

Day 4 — DSA: Two Pointers

Textbook-Style Detailed Notes

Two Pointers is one of the highest-value DSA patterns for Google-style interviews because it tests whether you can recognize structure in a problem and reduce unnecessary work.

The goal for today is not just to memorize two-pointer code. You should learn to answer:

Why are two pointers valid here, what invariant do they maintain, and why can I safely move one pointer without missing the answer?

1. What Is the Two-Pointer Technique?

The Two Pointer technique uses two indices/references to traverse a data structure—usually an array or string—in a coordinated way.

Instead of repeatedly searching the same elements, we maintain:

```
left  → ...
right → ...
```

and move one or both pointers according to the problem's conditions.

Typical forms:

Pattern A — Opposite directions

```
left → → →      ← ← ← right
[ 1  2  3  4  5  6 ]
  L                R
```

left starts at the beginning and right at the end.

Common for:

- Two Sum on sorted array
- Container With Most Water
- Palindrome
- 3Sum
- partitioning

Pattern B — Same direction

```
slow →
fast  →→
```

Both pointers start near the beginning but move at different rates.

Common for:

- Remove duplicates
- Move zeroes
- Fast/slow linked-list problems
- Partitioning
- In-place array modifications

Pattern C — Sliding-window style

```
left → [ window window window ] ← right
```

Two pointers define a variable-size range:

```
left = 0

for right in range(len(nums)):
    ...
    while condition_is_invalid:
        left += 1
```

This overlaps heavily with the sliding-window pattern, which you will study separately.

2. Why Two Pointers?

Suppose we have:

```
[1, 2, 3, 4, 5, 6]
```

and want to find whether two numbers sum to 7.

A brute-force solution checks:

```
1 + 2
1 + 3
1 + 4
1 + 5
1 + 6
2 + 3
...
```

That is:

$O(n^2)$

But because the array is sorted, we can exploit its structure.

Start:

```
[1, 2, 3, 4, 5, 6]
  L           R
```

Calculate:

```
1 + 6 = 7
```

We immediately know the answer.

The important point is:

The algorithm isn't fast because there are two variables called `left` and `right`. It is fast because the data has structure that allows us to eliminate many possibilities.

This is the fundamental idea behind two pointers.

3. The Most Important Question

Whenever you see a two-pointer problem, ask:

What information do I gain when I compare the elements at my two pointers?

For example:

```
nums[left] + nums[right]
```

If the array is sorted:

Sum too small

```
nums[left] + nums[right] < target
```

Then increasing right cannot help.

Why?

```
nums[right] is already the largest available candidate.
```

So we must increase left.

```
left++
```

Sum too large

```
nums[left] + nums[right] > target
```

Then increasing left cannot help because it would make the sum even larger.

So:

```
right--
```

This is the core reasoning.

4. Two Sum II — Classic Two Pointer Problem

Problem

Given a sorted array, find two numbers whose sum equals target.

Example:

```
nums = [2, 7, 11, 15]
target = 9
```

Answer:

```
2 + 7 = 9
```

Brute Force

```
def two_sum_brute(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]

    return []
```

Complexity:

```
Time: O(n2)
Space: O(1)
```

5. Two-Pointer Solution

```
def two_sum(nums, target):
    left = 0
    right = len(nums) - 1

    while left < right:
        current = nums[left] + nums[right]

        if current == target:
            return [left, right]

        elif current < target:
```

```

        left += 1
    else:
        right -= 1

    return []

```

Complexity:

Time: $O(n)$
 Space: $O(1)$

6. Dry Run

Given:

```

nums = [2, 7, 11, 15]
target = 9

```

Initial:

```

    L           R
    ↓           ↓
[2, 7, 11, 15]

```

Calculate:

$2 + 15 = 17$

Too large.

Therefore:

right--

Now:

```

    L           R
    ↓           ↓
[2, 7, 11, 15]

```

Calculate:

$2 + 11 = 13$

Still too large.

right--

Now:

```

    L     R
    ↓     ↓
[2, 7, 11, 15]

```

Calculate:

$2 + 7 = 9$

Found.

7. The Invariant

This is something you should learn to articulate during a Google interview.

An invariant is a condition that remains true throughout the algorithm.

For Two Sum on a sorted array:

At every iteration, all pairs that have been eliminated by pointer movement cannot contain a valid solution.

For example:

```
nums[left] + nums[right] > target
```

Because the array is sorted:

```
nums[left] ≤ nums[left+1] ≤ ... ≤ nums[right]
```

If the sum is too large, keeping right and increasing left would only make the sum larger.

Therefore, right can safely be discarded.

That's the correctness argument.

8. Why Sorted Order Matters

This is extremely important.

Two pointers aren't automatically valid just because you have an array.

Consider:

```
[8, 1, 7, 3]
```

Suppose:

```
target = 10
```

If we use:

```
left = 0  
right = 3
```

we get:

```
8 + 3 = 11
```

Could we safely move right?

No.

Because the array isn't sorted.

Maybe:

```
8 + 1 = 9
```

or:

```
7 + 3 = 10
```

The ordering doesn't give us enough information.

Therefore:

The pointer movement must be justified by a property of the data.

9. Two Pointer Recognition

You should start recognizing these clues.

Strong clues

Look for:

- sorted array
- sorted linked list
- pair/triplet
- palindrome
- target sum
- opposite ends
- remove duplicates
- in-place modification
- partitioning
- compare elements from both ends
- find a pair satisfying a condition

10. Opposite-End Template

Memorize the structure, not the exact code.

```
left = 0
right = len(nums) - 1

while left < right:

    # inspect nums[left] and nums[right]

    if condition:
        left += 1

    elif condition:
        right -= 1

    else:
        # found answer
```

You should be able to derive this during an interview.

11. Example — Valid Palindrome

Problem

Determine whether a string is a palindrome.

"racecar"

Compare:

```

r       r
e       e
c       c
a
```

```
def is_palindrome(s):
    left = 0
    right = len(s) - 1

    while left < right:
        if s[left] != s[right]:
            return False

        left += 1
        right -= 1
```

```
return True
```

Complexity:

```
Time: O(n)
Space: O(1)
```

12. Why Does This Work?

At every iteration we compare the corresponding characters:

```
s[left]
s[right]
```

For a palindrome:

```
first == last
second == second-last
...
```

If any pair differs:

```
not a palindrome
```

If every pair matches:

```
palindrome
```

The algorithm only needs to inspect half the string.

13. Example — Reverse Array In Place

Given:

```
[1, 2, 3, 4, 5]
```

We want:

```
[5, 4, 3, 2, 1]
```

Use two pointers:

```
def reverse_array(nums):
    left = 0
    right = len(nums) - 1

    while left < right:
        nums[left], nums[right] = nums[right], nums[left]

        left += 1
        right -= 1
```

Dry run:

```
[1, 2, 3, 4, 5]
L           R
```

swap

```
[5, 2, 3, 4, 1]
L           R
```

swap

```
[5, 4, 3, 2, 1]
L/R
```

Complexity:

```
Time: O(n)
Space: O(1)
```

14. Same-Direction Two Pointers

Now we move into another major form.

Consider:

```
[1, 1, 2, 2, 3]
```

Remove duplicates in-place.

We can maintain:

slow → position where next unique element should go

fast → scans the array

15. Remove Duplicates from Sorted Array

```
def remove_duplicates(nums):
    if not nums:
        return 0

    slow = 1

    for fast in range(1, len(nums)):
        if nums[fast] != nums[fast - 1]:
            nums[slow] = nums[fast]
            slow += 1

    return slow
```

Example:

```
[1, 1, 2, 2, 3]
```

Initially:

```
slow = 1
fast = 1
```

`nums[1] == nums[0]`

Skip.

Next:

```
fast = 2
```

Then:

```
nums[2] = 2
nums[1] = 1
```

Different.

Place:

```
nums[slow] = nums[fast]
```

Array becomes conceptually:

```
[1, 2, 2, 2, 3]
  ↑
  slow
```

Continue.

Final meaningful portion:

```
[1, 2, 3]
```

Return:

3

16. The Slow/Fast Mental Model

This is extremely useful.

```
FAST
↓
[1, 1, 2, 2, 3, 4, 4]
↑
SLOW
```

fast

Answers:

What have I discovered?

slow

Answers:

Where should I place the next valid element?

This pattern appears frequently in interviews.

17. Move Zeroes

Problem

Move all zeroes to the end while preserving the relative order of non-zero elements.

Input:

```
[0, 1, 0, 3, 12]
```

Output:

```
[1, 3, 12, 0, 0]
```

A two-pointer solution:

```
def move_zeroes(nums):
    slow = 0

    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
```

Here:

```
fast = scanner
slow = next position for a non-zero
```

18. Why Is This $O(n)$?

Because each pointer only moves forward.

Suppose:

```
left: 0 → 1 → 2 → ... → n
```

and:

right: $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots \rightarrow 0$

Even though there are two pointers, we're not necessarily doing nested loops.

The total movement is bounded by:

$O(n)$

This is an important interview concept.

Important distinction

Don't simply say:

"There are two pointers, so it's $O(n)$."

Instead explain:

"Each pointer moves monotonically and never moves backward, so across the entire execution there are $O(n)$ pointer movements."

That's a much stronger interview answer.

19. Two Pointers vs Nested Loops

Suppose you have:

```
for i in range(n):
    for j in range(n):
        ...
```

That's usually:

$O(n^2)$

But:

```
left = 0
right = n - 1

while left < right:
    ...
    left += 1
```

is:

$O(n)$

because the pointers move monotonically.

Important distinction

Don't simply say:

"There are two pointers, so it's $O(n)$."

Instead explain:

"Each pointer moves monotonically and never moves backward, so across the entire execution there are $O(n)$ pointer movements."

That's a much stronger interview answer.

20. Container With Most Water

This is one of the most important two-pointer problems.

Given:

```
[1, 8, 6, 2, 5, 4, 8, 3, 7]
```

Each value represents a vertical line.

Find two lines that contain the maximum amount of water.

Formula

For pointers left and right:

```
width = right - left
height = min(height[left], height[right])
```

Therefore:

Area = (right-left) × min(height[left], height[right])

21. The Key Insight

Suppose:

```
left height = 3
right height = 8
```

The water level is limited by:

3

Therefore:

```
Area = width × 3
```

If we move the right pointer inward:

width decreases

and the limiting height cannot become better because the left height remains 3.

So moving right is not useful.

Instead:

```
left++
```

This is the core proof.

Always move the pointer at the shorter line.

22. Implementation

```
def max_area(height):
    left = 0
    right = len(height) - 1
    best = 0

    while left < right:
        width = right - left
        current = width * min(height[left], height[right])

        best = max(best, current)

        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return best
```

Complexity:

Time: $O(n)$
Space: $O(1)$

23. 3Sum

Two pointers become even more powerful when combined with another pattern.

Problem

Find all unique triplets whose sum is zero.

Example:

```
[-1, 0, 1, 2, -1, -4]
```

Answer:

```
[-1, -1, 2]  
[-1, 0, 1]
```

Step 1 — Sort

```
[-4, -1, -1, 0, 1, 2]
```

Step 2 — Fix One Element

Suppose:

```
i = 0  
nums[i] = -4
```

Now solve:

```
two sum = 4
```

using two pointers.

```
      L           R  
      ↓           ↓  
[-4, -1, -1, 0, 1, 2]
```

General Structure

```
def three_sum(nums):  
    nums.sort()  
    result = []  
  
    for i in range(len(nums) - 2):  
  
        if i > 0 and nums[i] == nums[i - 1]:  
            continue  
  
        left = i + 1  
        right = len(nums) - 1  
  
        while left < right:  
            total = nums[i] + nums[left] + nums[right]  
  
            if total == 0:  
                result.append(  
                    [nums[i], nums[left], nums[right]]  
                )  
  
                left += 1  
                right -= 1  
  
            while left < right and nums[left] == nums[left - 1]:
```

```

        left += 1

        while left < right and nums[right] == nums[right + 1]:
            right -= 1

    elif total < 0:
        left += 1

    else:
        right -= 1

    return result

```

Complexity:

Sorting: $O(n \log n)$

Outer loop: $O(n)$

Two-pointer scan per iteration: $O(n)$

Total: $O(n^2)$

Space depends on whether we count the output and sorting implementation.

24. Why Sorting Is So Powerful

Sorting gives us monotonic information.

```
[-4, -1, 0, 2, 5, 8]
```

If:

```
nums[left] + nums[right] < target
```

then increasing right is not guaranteed to help—you already have the largest possible right value.

But increasing left moves toward larger values.

Therefore:

```
left++
```

is justified.

This is the fundamental relationship:

```

SORTED DATA
↓
MONOTONIC PROPERTY
↓
SAFE POINTER MOVEMENT
↓
ELIMINATION OF CANDIDATES
↓
O(n) OR O(n²) INSTEAD OF O(n²)/O(n³)

```

25. Two Pointers on Strings

Two pointers aren't restricted to arrays.

Example:

```
"abca"
```

Determine whether the string can become a palindrome after deleting at most one character.

The important idea:

```
left
```

right

Move inward while characters match.

When a mismatch occurs, try skipping either side.

Conceptually:

```
def valid_palindrome(s):
    left = 0
    right = len(s) - 1

    while left < right:
        if s[left] != s[right]:
            return (
                is_palindrome_range(s, left + 1, right)
                or
                is_palindrome_range(s, left, right - 1)
            )

        left += 1
        right -= 1

    return True
```

This demonstrates a more advanced pattern:

Two pointers + controlled branching.

26. Common Mistake #1 — Using Two Pointers Without Justification

Bad reasoning:

"This looks like a two-pointer problem."

Good reasoning:

"The array is sorted, so comparing the endpoints tells us whether we need a larger or smaller value. Therefore we can eliminate one candidate at each step."

Google interviews care about the second explanation.

27. Common Mistake #2 — Moving the Wrong Pointer

For:

```
nums[left] + nums[right] < target
```

move:

```
left++
```

For:

```
nums[left] + nums[right] > target
```

move:

```
right--
```

Because the array is sorted.

28. Common Mistake #3 — Forgetting `left < right`

For pair problems:

```
while left < right:
```

not:

```
while left <= right:
```

Usually we need two distinct elements.

29. Common Mistake #4 — Ignoring Duplicates

Especially important in:

- 3Sum
- 4Sum
- unique pairs
- unique triplets

Example:

```
[-1, -1, 0, 1]
```

If you don't handle duplicates, you may produce:

```
[-1, 0, 1]  
[-1, 0, 1]
```

when only one result is required.

30. Common Mistake #5 — Sorting Without Considering Original Indices

Suppose the problem asks:

Return original indices.

If you sort:

```
[3, 2, 4]
```

you lose the original positions unless you preserve them.

You may need:

```
indexed = [(value, index) for index, value in enumerate(nums)]  
indexed.sort()
```

Always check whether sorting changes something the problem requires you to preserve.

31. Two Pointers vs Sliding Window

These are related but shouldn't be mentally merged.

Two Pointers

Often:

```
left ↔ right
```

and the pointers move based on relationships between elements.

Example:

- two sum
- palindrome
- container with most water
- 3Sum

Sliding Window

Usually maintains a contiguous subarray/substring satisfying some condition.

Example:

- longest substring without repeating characters
- minimum window substring
- longest subarray with sum $\leq k$

Typical form:

```
left = 0

for right in range(n):
    add(nums[right])

    while window_invalid:
        remove(nums[left])
        left += 1
```

You'll study this as a separate pattern.

32. Two Pointers vs Binary Search

Another important distinction.

Binary Search

Reduces search space by roughly half:

```
n
↓
n/2
↓
n/4
↓
...
```

Complexity:

$O(\log n)$

Two Pointers

Usually scans through elements:

```
left →→→
right ←←←
```

Complexity:

$O(n)$

Both exploit structure, but differently.

33. Two Pointer Complexity Cheat Sheet

Problem | Typical Complexity

- Reverse array | $O(n)$
- Palindrome | $O(n)$
- Two Sum sorted | $O(n)$
- Remove duplicates | $O(n)$
- Move zeroes | $O(n)$
- Container With Most Water | $O(n)$
- 3Sum | $O(n^2)$
- 4Sum | $O(n^3)$ typically
- Sort + two pointer | $O(n \log n)$
- Linked-list fast/slow | $O(n)$

34. The Interview Framework

When you identify two pointers, explain your solution in this order.

Step 1 — State the brute force

"The straightforward approach considers every pair, which takes $O(n^2)$."

Step 2 — Identify the exploitable structure

"The array is sorted."

Step 3 — Define pointers

"I'll place one pointer at the beginning and one at the end."

Step 4 — Explain movement

"If the current sum is smaller than the target, I move the left pointer because that is the only way to increase the sum. If it is larger, I move the right pointer."

Step 5 — Explain correctness

"Because the array is sorted, the pointer movement safely eliminates candidates."

Step 6 — Complexity

"Each pointer moves at most n positions, giving $O(n)$ time and $O(1)$ auxiliary space."

This is the level of explanation you should practice.

35. Two Pointer Mental Checklist

When reading a new problem, ask:

- 1. Is the data sorted?
- 2. Am I looking for a pair/triplet?
- 3. Can I compare elements from opposite ends?
- 4. Can pointer movement eliminate candidates?

- 5. What happens if the current value is: too small? too large? equal?
- 6. Can each pointer move only in one direction?
- 7. What invariant makes the movement safe?
- 8. Are duplicates involved?
- 9. Do I need original indices?
- 10. Does sorting change something important?

If you can answer these, you are actually recognizing the pattern, rather than memorizing it.

36. Google-Level Insight

The deeper concept behind Two Pointers is:

Monotonic elimination.

You're repeatedly proving:

These candidates cannot possibly produce the answer.

For example:

Sorted array:

```
[1, 2, 3, 4, 5, 6]
  L             R
```

If:

$1 + 6 < \text{target}$

then 6 cannot pair with 1.

But that's not enough to eliminate 6 entirely—you need to reason carefully about which pointer movement is safe.

If:

$1 + 6 > \text{target}$

then 1 cannot pair with 6 or anything larger than 6 (there is nothing larger at this endpoint), and with the current 1, the right endpoint is too large. Moving right inward is safe.

The ability to make these candidate-elimination arguments is what makes the technique powerful.

37. What You Should Be Able to Code Today

For Day 4, don't just read these.

Implement from memory:

Level 1 — Fundamental

- 1. Reverse Array
- 2. Valid Palindrome
- 3. Two Sum II — Sorted Array

Level 2 — Same Direction

- 4. Remove Duplicates from Sorted Array

- 5. Move Zeroes

Level 3 — Interview

- 6. Container With Most Water
- 7. 3Sum

Level 4 — Reasoning

- 8. Valid Palindrome II
- 9. Squares of a Sorted Array
- 10. Merge Two Sorted Arrays

38. Day 4 DSA Definition of Done

You can consider the Two Pointer portion of Day 4 complete only when you can do all of the following without looking at notes:

- Explain what Two Pointers is.
- Explain opposite-direction pointers.
- Explain same-direction pointers.
- Explain slow/fast pointers.
- Recognize when sorting enables Two Pointers.
- Explain why pointer movement is safe.
- State the invariant.
- Implement Two Sum II.
- Implement Valid Palindrome.
- Implement Reverse Array.
- Implement Remove Duplicates.
- Implement Move Zeroes.
- Implement Container With Most Water.
- Implement 3Sum.
- Handle duplicates correctly.
- Explain time and space complexity.
- Explain why two pointers beats brute force.
- Distinguish Two Pointers from Sliding Window.
- Distinguish Two Pointers from Binary Search.
- Solve a new two-pointer problem by identifying the invariant rather than recalling a template.

The one sentence I want you to remember

Two Pointers works when the problem gives you enough structure to move a pointer while safely eliminating a set of candidates.

That principle will matter much more in your Google interviews than memorizing the code template.